



TeleAgent

LeetCode Bug

2026-09-06

一、统一模板

所有回溯题的骨架完全一致，区别仅在于五个关键变量的选择。

```
void back(参数, int start, List<...> path) {  
    if (终止条件) {  
        res.add(new ArrayList<>(path)); // 存快照  
        return;  
    }  
    for (int i = start; i < n; i++) {  
        if (需要去重) continue;  
        if (需要剪枝) continue/break;  
        path.add(选元素);  
        back(参数, 下一层start, path);  
        path.remove(path.size() - 1); // 回溯  
    }  
}
```

二、五个关键变量

1. start：控制从哪开始选

传参	含义	代表题
i + 1	每个元素只选一次	77、216、40、78、90
i	可重复选同一元素	39
不用 start	全排列，每次从头选	46
不用 start，用 vis	带"用过"标记	46、37、51

2. 收集结果的时机

题型	何时收集	代表题
组合/分割	满足终止条件时	39、40、77、216、131、93
子集	每进入一次递归就收集	78、90
子序列	长度 ≥ 2 时收集	491
棋盘类	每行放完到最后一行	51、37

3. 去重：两种方式

方式	适用场景	代表题	易错点
排序 + i > start && nums[i] == nums[i-1]	可以排序的题	40、90	不能漏 i > start
Set 每层新建	不能排序/保持原序	491	491 用 nums[i]==nums[i-1] 会漏掉非相邻重复

4. 剪枝

方式	代表题	条件
----	-----	----

排序 + break	39、216	sum + candidates[i] > target, 后面更大不可能
不剪	77、78、46	数据量小, 不需要

5. new ArrayList<>(path): 存快照不是存引用

所有题通用，不加这个全是空列表。res.add(prev) 存的是引用，prev 后续被修改后 res 里的元素也跟着变，最终全是空列表。new ArrayList<>(prev) 拍当前状态快照，之后 prev 怎么改都不影响。

三、两个进阶（棋盘类题目引入）

进阶 1：返回值——找全部 vs 找一个

这是 37（数独）相比之前所有题最大的区别。

返回类型	含义	代表题	撤销逻辑
void	找所有解	39、77、46、51、491...	递归后总是撤销
boolean	找一个解就停	37（数独）	递归返回 true 就不撤销，直接传上去

数独的关键结构：递归返回 `true` 说明找到解了，自己什么也不做直接返回 `true`，不执行撤销，保住解。1~9 全试过都不行，`return false` 告诉上层换一个数。

```
if (back(...)) return true; // 找到了，不撤销，一路传 true
// 走到这里说明没找到，撤销，试下一个
board[i][j] = '.';
```

进阶 2：vis 数组——标记冲突

N 皇后和数独都需要标记"哪个位置不能放"，但设计方式不同。

方案	代表题	优点	缺点
二维 vis 网格 + 计数器(++/--)	51（皇后的原始思路）	直观	代码量大，6 段循环标记+撤销
三个一维布尔数组	51、37	一行标记一行撤销， $O(1)$	需要理解对角线映射
三个二维 int 数组	37（数独）	查询 $O(1)$	下标设计容易出错

N 皇后的对角线映射技巧

```
boolean[] col = new boolean[n]; // 列冲突
boolean[] diag1 = new boolean[2 * n]; // 主对角线: row + col 相同
boolean[] diag2 = new boolean[2 * n]; // 副对角线: row - col + n - 1 相同
```

数独的宫映射技巧

```
int box = (i / 3) * 3 + j / 3; // 行列坐标 → 第几个宫
```

四、典型 Bug 回顾

Bug	出现的题	根因
res.add(prev) 存引用而非快照	77	同一个 List 对象被反复修改
prev.delete(i) 方法不存在	77	Java List 没有 delete, 用 remove(size-1)
递归传 start+1 非 i+1	77、216	start 是本层起点, i 才是当前选中元素
nums[i]==nums[i-1] 用于未排序数组	491	只能检测相邻重复, 非相邻的漏掉
for 循环条件里放 vis[i]==0	46	遇到第一个已访问就退出循环, 应放循环体里 continue
setCharAt 返回 void 不能链式调用	51	和 append 不同, 不能接 .toString()
card 下标少除 3	37	$i/3*3+j$ 应为 $i/3*3+j/3$
back 返回 void 解被撤销	37	找到解后无条件回溯, 解被擦掉
back 缺 return 语句	37	boolean 方法所有路径都要有返回值

五、一句话心法

回溯就是一棵多叉树的 DFS。每道题只需想清楚四件事：

1. 每层循环选什么（选数字？选字母？选列位置？填1~9？）
2. 什么时候收集/停止（满足条件？每个节点？找到就停？）
3. 怎么避免重复（start 控顺序，去重控同值，vis 标冲突）
4. 撤销要和标记严格对应（加了多少减多少，标记了什么撤销什么）